

Reliability Checks for Machine-Learning Surrogates in Scientific Software

Applicant: Isaac Malsky, NASA Postdoctoral Program Fellow
Affiliation: Jet Propulsion Laboratory, California Institute of Technology
Research track: AI/ML & Scientific Software

Project Goals and Objectives

Surrogate models are ML frameworks that approximate the input-to-output mapping of classical, deterministic physical solvers [1]. By training an ML model on examples from a solver, a surrogate model can reproduce the solver’s behavior at a fraction of the computational cost while asymptoting to its accuracy. These models are now used across scientific software. Neural operators learn solution maps of partial differential equations [12, 13]; weather and climate models use learned components for global prediction [9, 10, 11]; and domain teams apply emulators to chemistry, radiative transfer, materials, and inverse problems. **These ML methods open capabilities that were not previously possible:** higher model resolutions, more complete physics, large uncertainty-quantification studies, and differentiable simulations [12, 13, 9, 10, 11].

However, checks on the reliability of these surrogate models are not standardized, and are often not sufficient. Reliability is usually judged by a held-out test-set error, a wall-clock speedup, and a handful of example plots. These checks are necessary, but not sufficient. A model with low average error, as in Figure 1, can still fail in specific regions of parameter space, silently extrapolate, or violate known physical constraints. For example, normalization leakage is one common and difficult-to-see failure. A recent analysis of ML workflows across 17 scientific fields identified 294 papers whose reported performance was inflated by data leakage during preprocessing [8].

What is missing is a computationally practical and standardized way to test the reliability of ML surrogates before they are incorporated within larger scientific software systems. Prominent efforts such as NeuralGCM [9], GraphCast [10], ACE2 [11], CliMA [6], and PhysicsNeMo [5] show the scale of recent progress in ML surrogate modeling. However, validation tooling has not kept up. **No existing toolkit directly answers the practical reliability questions that arise when a neural emulator becomes part of a scientific software stack.**

This project addresses this gap: we will create a lightweight validation package for scientific ML surrogates. Every emulator author has to make design choices that strongly shape the final model. For example, what specific normalization method should be used for each channel? How should the loss weigh outputs that span different units, scales, or levels of scientific importance? Should the model enforce constraints such as conservation, positivity, symmetry, monotonicity, or known bounds [15, 16]? **The community currently has no consolidated way to test or visualize the consequences of these choices.**

We have three primary objectives:

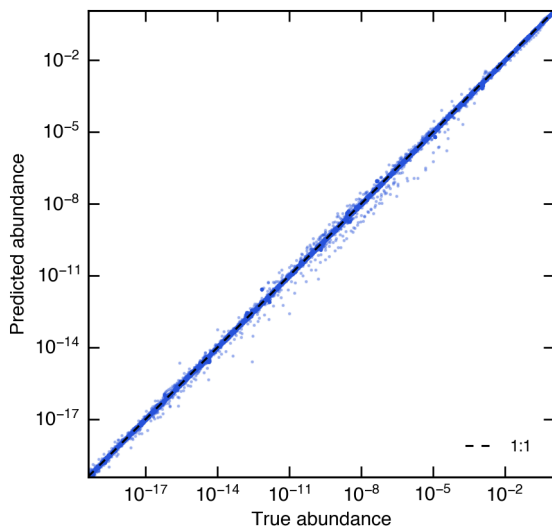


Figure 1: Aggregate accuracy for a representative surrogate can hide extrapolation and preprocessing failures. A poor understanding of the accuracy can hide specific model failures.

1. Create an open-source Python package that tests normalization, data-split hygiene, training-domain coverage, constraint violations, calibration, exported inference, and speed.
2. Validate the package on representative local, sequential, and spatial-field emulators.
3. Open-source the package, documentation, and tutorial material for community use.

Expected Impact on Scientific Software Community

Although slow, classical solvers are built from our understanding of the true underlying physics. The assumptions and failure modes of surrogate models are often harder to diagnose. Without a validation workflow, groups publish models that are difficult to reproduce, compare, or reuse outside the notebook in which they were trained. The direct deliverable is a small Python package and report template to show common surrogate model validation checks. This matters in any domain where ML surrogates are used within larger modeling pipelines, including climate, chemistry, materials science, and fluid dynamics simulations. As emulators become more common, the community needs a shared set of tools for what has been tested and where each model should not be trusted.

Implementation Plan

Diagnostic checks: The package will implement checks drawn from common failure modes in scientific ML workflows and turn them into clear validation reports that can be attached to a model release. The core diagnostics will test normalization and train/test split leakage [8]; compare training, validation, and test distributions with histograms, density plots, quantile–quantile plots, and coverage maps; and show errors with predicted-vs.-true plots, residual histograms, per-channel tables, and field-level maps like Figure 2. The package will also include simple tests for physical constraints, model export, and inference speed, so users can quickly see whether a surrogate is accurate, reproducible, and fast enough for its intended use.

Representative examples: We will validate the package against three small surrogate models that are representative of common scientific ML workflows. For a local state-to-state emulator, we will use the Robertson stiff chemical-kinetics problem, a canonical three-species ODE benchmark with widely separated reaction time scales [21]. For an ordered-output emulator, we will use ODE trajectory benchmarks such as the Lorenz system, where the surrogate predicts a time-ordered state trajectory sampled on a fixed output grid [22]. For a spatial-field emulator, we will use standard Fourier neural operator benchmarks, such as Burgers’ equation, Darcy flow, and the Navier–Stokes equation [12, 13]. These examples are small enough to run and document within the fellowship, but broad enough to showcase the validation setup.

Demonstration of feasibility: This project is well suited to a six-month URSSI fellowship. The work is not to build a scientific model, but to make the validation layer that emulator authors currently have to recreate for each project. We will run the package on all examples and publish the resulting reports as templates that other groups can adapt for their own surrogate models.

Community Engagement Strategy

Different scientific domains will need different reliability checks, so the diagnostic set must be shaped by community feedback. We will host the package on GitHub from day one, with monthly progress reports, tagged issues for community feedback, and example reports attached to releases. We will reach out to

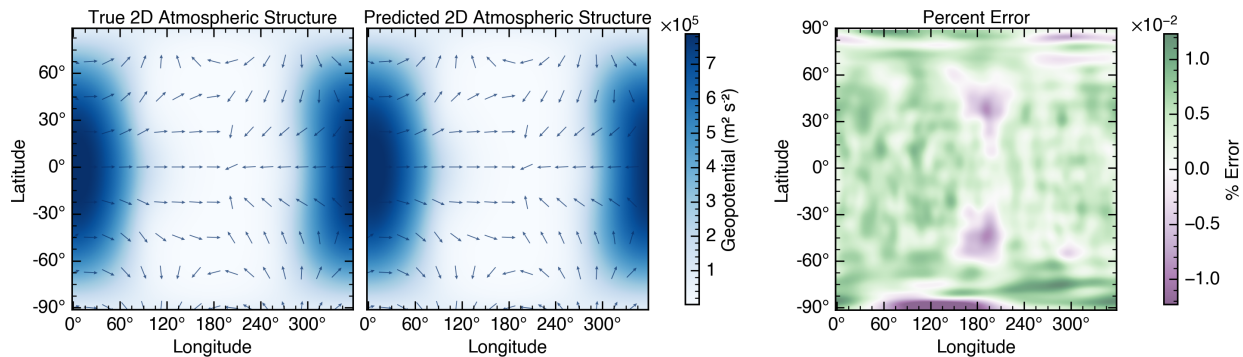


Figure 2: Example diagnostic: true solver value, predicted value, and percent-error map. Different forms of visualizations are critical to understand model parameter space coverage.

researchers building ML emulators in climate and weather, computational chemistry, materials science, fluid dynamics, and other simulation-heavy fields to identify checks that are missing, too narrow, or too expensive for routine use. JPL hosts an active AI/ML research community that can provide early review from researchers working across multiple scientific software stacks. We will contribute those lessons back to the URSSI community through a short case study on one common failure mode: how normalization leakage can make a surrogate appear accurate while producing a model that cannot be reproduced or trusted outside its original training workflow.

Evaluation Metrics

Quantitative: The project will be successful if we produce a package for diagnostic checks and model validation reports, tagged releases of the package and documentation, and one URSSI newsletter or blog contribution.

Qualitative: The reports should make assumptions, unsupported regions, and failure modes legible to a non-author. Community feedback should lead to documented changes in the diagnostic set or interface, and the final guidance should apply cleanly across at least two scientific domains.

Timeline and Deliverables

Month	Activities and deliverables
1 (Jun–Jul)	Define validation contract; publish package skeleton.
2 (Jul–Aug)	Implement split, normalization, distribution, and export checks.
3 (Aug–Sep)	Add error, calibration, constraint, and speed diagnostics.
4 (Sep–Nov)	Validate examples; publish workflow, tutorial, and release.

Final deliverables: Python package; three validation reports; reusable workflow; tutorial; archived examples; URSSI case study.

References

- [1] Brunton, S. L. and Kutz, J. N. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2022.
- [2] Deepchecks contributors. Deepchecks: tests for continuous validation of ML models and data. <https://github.com/deepchecks/deepchecks>.
- [3] Groen, D., Arabnejad, H., Jancauskas, V., et al. VECMAtk: a scalable verification, validation and uncertainty quantification toolkit for scientific simulations. *Philosophical Transactions of the Royal Society A*, 379, 20200221, 2021.
- [4] Brei, N., Mei, X., Lersch, D., Rajput, K., and Lawrence, D. PHASM: a toolkit for creating AI surrogate models within legacy codebases. Technical Report, Jefferson Lab, 2024.
- [5] NVIDIA Corporation. PhysicsNeMo: an open-source framework for physics-AI models. <https://github.com/NVIDIA/physicsnemo>.
- [6] Climate Modeling Alliance. CliMA: a new climate model built from the ground up. <https://clima.caltech.edu/>.
- [7] Pouchard, L., Reyes, K. G., Alexander, F. J., and Yoon, B.-J. A rigorous uncertainty-aware quantification framework is essential for reproducible and replicable machine learning workflows. *Digital Discovery*, 2, 1210–1224, 2023.
- [8] Kapoor, S. and Narayanan, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4, 100804, 2023.
- [9] Kochkov, D., Yuval, J., Langmore, I., et al. Neural general circulation models for weather and climate. *Nature*, 632, 1060–1066, 2024.
- [10] Lam, R., Sanchez-Gonzalez, A., Willson, M., et al. Learning skillful medium-range global weather forecasting. *Science*, 382, 1416–1421, 2023.
- [11] Watt-Meyer, O., Henn, B., McGibbon, J., et al. ACE2: accurately learning subseasonal to decadal atmospheric variability and forced responses. arXiv:2411.11268, 2024.
- [12] Li, Z., Kovachki, N., Azizzadenesheli, K., et al. Fourier neural operator for parametric partial differential equations. *Proceedings of the International Conference on Learning Representations*, 2021.
- [13] Kovachki, N., Li, Z., Liu, B., et al. Neural operator: learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24, 1–97, 2023.
- [14] Bonev, B., Kurth, T., Hundt, C., et al. Spherical Fourier neural operators: learning stable dynamics on the sphere. *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [15] Beucler, T., Pritchard, M., Rasp, S., Ott, J., Baldi, P., and Gentine, P. Achieving conservation of energy in neural network emulators for climate modeling. arXiv:1906.06622, 2019.
- [16] Hansen, D., Maddix, D. C., Alizadeh, S., et al. Harnessing the power of neural operators with automatically encoded conservation laws. arXiv:2312.11176, 2023.

- [17] Angelopoulos, A. N. and Bates, S. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv:2107.07511, 2021.
- [18] Tahseen, T., et al. Machine-learning-accelerated radiative transfer in a Venus general circulation model. 2024.
- [19] Malsky, I., et al. A transformer surrogate for multiwavelength radiative transfer in planetary atmospheres. In preparation, 2025.
- [20] Malsky, I., Zhang, X., Kataria, T., et al. Accelerating chemical kinetics for exoplanet atmospheres using neural networks. Submitted, 2025. Code: <https://github.com/imalsky/Chemulator>.
- [21] Robertson, H. H. The solution of a set of reaction rate equations. In *Numerical Analysis: An Introduction*, pp. 178–182. Academic Press, 1966.
- [22] Lorenz, E. N. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20, 130–141, 1963.